Rockchip Android 16.0 SSI SDK开发指南

文件标识：RK-KF-YF-E66

发布版本：V1.1.0

日期：2025-010-13

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文介绍Rockchip Android16 SDK的开发，帮助开发者快速入门。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	吴良清	2025-7-24	发布支持RK3576/RK3562的RKR1版本的SSI SDK
V1.1.0	吴良清	2025-10-13	发布支持RK3588系列/RK3568系列/RK356系列/RK3326系列/PX30系列/RK3399的RKR3版本的SSI SDK

文档问题反馈：wlq@rock-chips.com

目录

Rockchip Android 16.0 SSI SDK开发指南

1. 名词解释
2. Rockchip Android 16.0 SSI SDK说明
3. Rockchip Android 16.0 SSI SDK支持芯片
4. Rockchip Android 16.0 SSI SDK代码下载编译
   - 4.1 代码下载
     - 4.1.1 下载地址
     - 4.1.2 服务器镜像下载
   - 4.2 搭建自己的repo代码服务器
     - 4.2.1 环境
     - 4.2.2 gitolite搭建
       - 4.2.2.1 服务器端操作
       - 4.2.2.2 客户端操作
     - 4.2.3 repo镜像搭建
       - 4.2.3.1 服务器端操作
       - 4.2.3.2 客户端操作
     - 4.2.4 客户端操作
   - 4.3 代码管理
     - 4.3.1 切换自己的代码分支
     - 4.3.2 代码修改提交
     - 4.3.3 同步RK的代码
   - 4.4 kernel代码路径说明
   - 4.5 SSI SDK编译
     - 4.5.1 编译条件
     - 4.5.2 编译说明
     - 4.5.3 如何关闭GKI
     - 4.5.4 编译工具rkbuild
     - 4.5.5 编译步骤
     - 4.5.6 固件说明
   - 4.6 固件烧写
     - 4.6.1 固件烧写工具
   - 4.7 Generic Kernel Image（GKI）
   - 4.8 fastboot烧写动态分区
5. 使用DTBO功能
6. 修改fstab文件
7. 修改parameter.txt
8. Android14 SDK
9. 附录 A 编译开发环境搭建 Compiling and development environment setup
   - 9.1 Initializing a Build Environment
   - 9.2 Choosing a Branch
   - 9.3 Setting up a Linux build environment
   - 9.4 Installing the JDK
   - 9.5 Configuring USB Access
10. 附录 B SSH公钥操作说明 SSH public key operation instruction
    - 10.1 附录 B-1 SSH公钥生成 SSH public key generation
    - 10.2 附录 B-2 使用key-chain管理密钥 Use key-chain to manage the key
    - 10.3 附录 B-3 多台机器使用相同ssh公钥 Multiple devices use the same ssh public key
    - 10.4 附录 B-4 一台机器切换不同ssh公钥 Switch different ssh public keys on one device
    - 10.5 附录 B-5 密钥权限管理 Key authority management
    - 10.6 附录 B-6 Git权限申请说明 Git authority application instruction

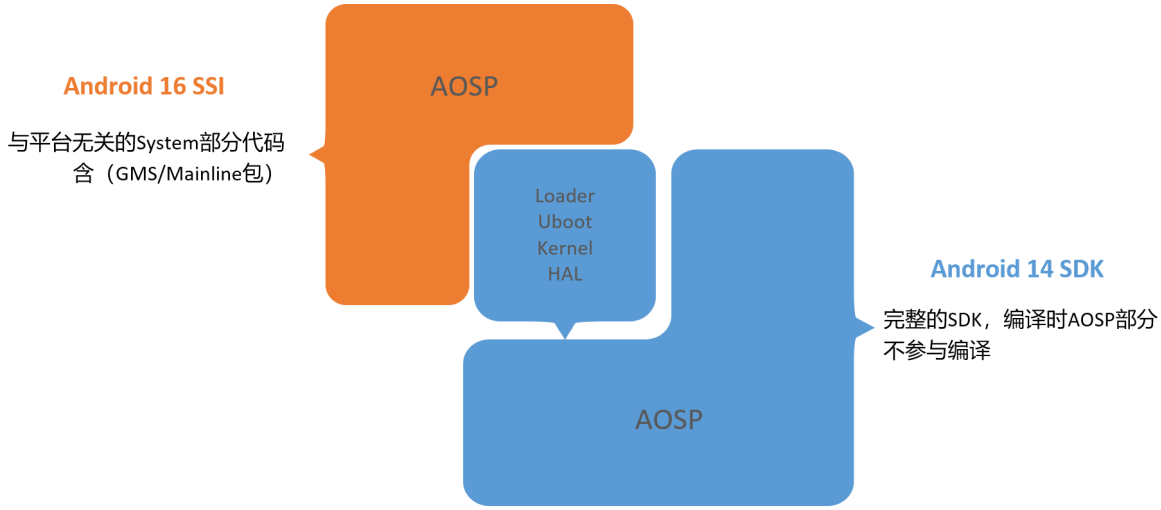
1. 名词解释

- SSI （Shared System Image）共享系统镜像

2. Rockchip Android 16.0 SSI SDK说明

- Rockchip Android 16.0 SSI SDK 仅发布aosp的代码，编译后仅生成system相关的镜像，如system.img、system_ext.img、product.img，和google的GSI的固件是一样的。其他部分的代码需要使用Android14的SDK来编译，所以Rockchip Android 16.0 SSI SDK需要搭配Android14 的SDK使用。

-- Rockchip Android 16.0 SSI SDK与Android14 SDK组合成完整的SDK的架构框图如下：



- Rockchip Android 16.0 SSI SDK适配所有芯片平台（需要Android14更新支持），即同一个system.img可以兼容所有已经支持的芯片平台，如RK3576/RK3562等。
- SSI的方式编译方法与其他SDK不同，我们重新开发一套编译工具rkbuild，在下面编译章节详细介绍。

3. Rockchip Android 16.0 SSI SDK支持芯片

SSI的system固件支持所有芯片，需要Android14 SDK更新到对应的版本，Android14 SDK版本对Android16 SSI支持情况如下：

芯片平台	是否支持	**Android14 SDK版本**	**Android16 SSI SDK版本**
RK3562	支持	Android14 RKR7	Android16 RKR1
RK3576系列	支持	Android14 RKR7	Android16 RKR2
RK3588系列	支持	Android14 RKR8	Android16 RKR3
RK3568系列	支持	Android14 RKR8	Android16 RKR3
RK3566系列	支持	Android14 RKR8	Android16 RKR3
RK3326系列	支持	Android14 RKR8	Android16 RKR3
PX30系列	支持	Android14 RKR8	Android16 RKR3
RK3399系列	支持	Android14 RKR8	Android16 RKR3

4. Rockchip Android 16.0 SSI SDK代码下载编译

4.1 代码下载

4.1.1 下载地址

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI.xml
```

如果是要过GMS/EDLA认证，且申请的是Express版本的SDK（需要有google的MADA协议才能申请），那么下载地址如下：

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI_Express.xml
```

4.1.2 服务器镜像下载

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI.xml --mirror
```

如果是要过GMS/EDLA认证，且申请的是Express版本的SDK（需要有google的MADA协议才能申请），那么下载地址如下：

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI_Express.xml --mirror
```

注，repo是google用Python脚本写的调用git的一个脚本，主要是用来下载、管理Android项目的软件仓库，其下载地址如下：

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

为方便客户快速获取SDK源码，瑞芯微技术窗口通常会提供对应版本的SDK初始压缩包。以Rockchip_Android16.0_SDK_RELEASE.tar.gz.*`为例，拷贝到该初始化包后，通过如下命令可检出源码：

```
mkdir Rockchip_Android16.0_SDK_RELEASE
cat Rockchip_Android16.0_SDK_RELEASE.tar.gz* | tar -zx -C
Rockchip_Android16.0_SDK_RELEASE
cd Rockchip_Android16.0_SDK_RELEASE
.repo/repo/repo sync -l
.repo/repo/repo sync -c
```

4.2 搭建自己的repo代码服务器

4.2.1 环境

安装`openssh-server`用于远程登录，`git`用于管理工程，`keychain`用于公私钥管理工具

```
sudo apt-get install openssh-server git keychain
```

4.2.2 gitolite搭建

4.2.2.1 服务器端操作

（以服务器地址：10.10.10.206为例进行说明）

1. 创建git账户：

```
sudo adduser --system --shell /bin/bash --group git
sudo passwd git
```

2. 以“git”账户登录服务器
3. 确保“~/.ssh/authorized_keys”为空或者不存在
4. 拷贝服务器管理员的公钥到“~/YourName.pub”
5. 下载gitolite源码

```
git clone https://github.com/sitaramc/gitolite.git
```

6. 在git用户目录下创建bin目录

```
mkdir -p ~/bin
```

7. 执行下列命令安装gitolite，不同版本安装方法不同，请参考源码中的文档：

```
gitolite/install -to ~/bin
```

8. 设置管理员

```
~/bin/gitolite setup -pk YourName.pub
```

4.2.2.2 客户端操作

1. 克隆服务器的gitolite管理仓库：

```
git clone ssh://git@10.10.10.206/gitolite-admin.git
```

2. 添加用户公钥到gitolite目录下

```
cp username.pub keydir/username.pub
```

3. 添加管理员用户

```
vi conf/gitolite.conf
@admin = admin1 admin2 admin3
repo gitolite-admin
RW+     =   @admin
```

4.2.3 repo镜像搭建

4.2.3.1 服务器端操作

1. 用git账号登入服务器
2. 在根目录下载repo工具

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

3. 新建RK_Android16_mirror目录

```
mkdir RK_Android16_mirror
```

4. 进入 RK_Android16_mirror目录

```
cd RK_Android16_mirror
```

5. 下载RK Android16 SDK镜像

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -m A16_SSI.xml --
mirror
```

6. 创建仓库组权限

```
.repo/repo/repo list -n > Android_16.conf
sed -i 's/^/@Android_16 = RK_Android16_mirror\/&/g' Android_16.conf
```

4.2.3.2 客户端操作

1. 将服务器端的Android_16.conf拷贝到客户端的·gitolite-admin/conf/·下
2. 添加组权限

```
vi conf/Android_16.conf
@usergroup = user1 user2 user3
repo @Android_16
R   = @usergroup
RW+ = @admin
```

```
vi conf/gitolite.conf
include "Android_16.conf"
```

3. 新建自己的manifests仓库

```
vi conf/Android_16.conf
@Android_16 = Android_T/manifests_xxx
```

4.2.4 客户端操作

1. 在客户端下载manifests_xxx仓库

在其他客户端电脑上下载manifests_xxx.git仓库

```
git clone ssh://git@10.10.10.206/Android_16/manifests_xxx.git
```

2. 在客户端下载原始manifests仓库

```
git clone ssh://git@10.10.10.206/Android_16/manifests.git
```

3. 提交manifest.xml文件到新建的manifest_xxx仓库中

将原始manifests下面的文件拷贝到的manifests_xxx内

```
cd manifests_xxx
cp -rf manifests/*.xml manifests_xxx/
```

查看拷贝文件

```
git status

 Android16.xml
 Android16_Express.xml
 default.xml
 include/rk3562_repository.xml
 include/rk_checkout_from_aosp.xml
 include/rk_modules_repository.xml
 remote.xml
 remove_u.xml
 remove_unused.xml
```

本地提交

```
git add -A
git commit -m "init xxx"
```

push到远程分支

```
git push origin master:master
```

4. 创建自己的代码下载链接

在根目录下下载repo工具

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

按以上步骤操作后，自己的代码下载链接如下

```
mkdir Android16
cd Android16
~/repo/repo init -u ssh://git@10.10.10.206/Android_16/manifests_xxx.git -m
Android16.xml
```

其中：
`//10.10.10.206` 是你的服务器端地址

通过以上步骤就可以完成自己的repo服务器搭建了，可以把自己的代码服务器链接分享给同事们一起工作了。

4.3 代码管理

通过以上步骤搭建代码服务器后大部分代码仓库都使用RK默认的分支，如果有仓库需要修改自己的代码，可以参考下面的步骤进行操作。

4.3.1 切换自己的代码分支

1. 进入需要修改的代码仓库，以kernel目录为例进行说明

```
cd kernel-6.1
```

2. 切换一个本地分支

```
git checkout remotes/m/master -b xxx_branch
```

3. push xxx_branch分支到远程服务器

```
git push rk29 xxx_branch:xxx_branch
```

其中`rk29`是remote 可以直接tab键自动补全

4. 进入.repo/manifests目录修改manifest里面指定的分支

进入.repo/manifests目录通过grep kernel可以找到kernel仓库对应的manifest的位置

```
cd .repo/manifests
```

```
--- a/include/rk_modules_repository.xml
+++ b/include/rk_modules_repository.xml
@@ -10,7 +10,7 @@
   <project path="hardware/rockchip/libgraphicpolicy"
name="rk/hardware/rk29/libgraphicpolicy" remote="rk"
revision="refs/tags/android-1s.0-mid-rkr1" />
   <project path="hardware/rockchip/libhwjpeg" name="rk/hardware/rk29/libhwjpeg"
remote="rk" revision="refs/tags/android-16.0-mid-rkr1"/>
   <project path="u-boot" name="rk/u-boot" remote="rk"
revision="refs/tags/android-16.0-mid-rkr1"/>

-  <project path="kernel" name="rk/kernel" remote="rk29"
revision="refs/tags/android-16.0-mid-rkr1"/>

+  <project path="kernel" name="rk/kernel" remote="rk29" revision="xxx_branch"/>

   <project path="bootable/recovery/rkupdate"
name="platform/bootable/recovery/rk_update" remote="rk"
revision="refs/tags/android-16.0-mid-rkr1"/>
   <project path="bootable/recovery/rkutility"
name="platform/bootable/recovery/rk_utility" remote="rk"
revision="refs/tags/android-16.0-mid-rkr1"/>
```

5. 提交修改的manifest到远程分支

```
git add include/rk_modules_repository.xml
git commit -m "change kernel branch on xxx_branch"
git push origin default:master
```

提交manifests仓库后，其他同事就可以同步到你们自己的分支的kernel代码了。

4.3.2 代码修改提交

按上面步骤切换完分支后就可以在自己分支上提交自己的修改了， 提交直接push到xxx_branch分支上面。

4.3.3 同步RK的代码

1. 同步RK代码需要在服务器端进行sync操作

```
cd RK_Android16_mirror
```

```
.repo/repo/repo sync -c
```

2. 客户端合并RK对manifests的修改

- 下载RK原始manifests仓库

```
git clone //10.10.10.206/wlq/test/manifests.git
```

使用对比工具对比manifests（RK原始）和manifests_xxx(自己的)，将RK修改的差异部分合并到自己的仓库中（主要修改tag，增加删除仓库等）。

- 对比确认后将修改push到manifests_xxx上。

这步也可以确认自己修改了哪些仓库，在下一步中将进行修改仓库的合并。

3. 有自己切分支的目录需要手动把RK的修改merge到自己的分支上面

以kernel为例：

- 查看当前指向的远程分支

```
wlq@wlq:~/home1/test2/kernel-6.1$ git branch -av
* android-11.0-mid-rkr7   0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
  xxx_branch              0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
  remotes/m/master        -> rk29/xxx_branch
  remotes/rk29/xxx_branch 0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
```

可以看到当前指向的是：`remotes/m/master        -> rk29/xxx_branch`

- 创建本地分支（从自己的远程分支上切）

```
git checkout remotes/m/xxx_branch -b local_xxx_branch
```

- 确认当前RK发布的最新TAG

```
wlq@wlq:~/home1/test2/kernel$ git tag | grep rkr
android-10.0-mid-rkr1
android-10.0-mid-rkr10
android-10.0-mid-rkr11
android-10.0-mid-rkr13
android-10.0-mid-rkr2
android-10.0-mid-rkr3
android-10.0-mid-rkr4
android-10.0-mid-rkr5
android-10.0-mid-rkr6
android-10.0-mid-rkr7
```

```
android-10.0-mid-rkr8
android-10.0-mid-rkr9
android-11.0-ebook-rkr1
android-11.0-ebook-rkr2
android-11.0-ebook-rkr3
android-11.0-ebook-rkr4
android-11.0-ebook-rkr5
android-11.0-ebook-rkr6
android-11.0-mid-rkr1
android-11.0-mid-rkr2
android-11.0-mid-rkr3
android-11.0-mid-rkr4
android-11.0-mid-rkr4.1
android-11.0-mid-rkr5
android-11.0-mid-rkr6
android-11.0-mid-rkr7
android-11.0-mid-rkr7-prev
android-11.0-mid-rkr8
android-16.0-mid-rkr1
```

可以看到当前最新的Android16的tag是`android-16.0-mid-rkr1`

- 合并`android-16.0-mid-rkr1`到本地分支

```
git merge android-16.0-mid-rkr1
```

查看是否有冲突，如果有冲突先解决冲突，没有冲突在执行下一步

- push合并完的代码到远程分支

```
git push rk29 local_xxx_branch:xxx_branch
```

- 其他切分的目录都按这个方式进行合并提交即可

4.4 kernel代码路径说明

Android16支持6.1 版本的kernel，kernel源码在工程中kernel-6.1目录下，

4.5 SSI SDK编译

4.5.1 编译条件

需要有**Android14的SDK**，且**Android14的SDK版本要是RKR7及以上版本**。编译的时候会需要链接到Android14的SDK。如果没有Android14的SDK则需要先下载Android14 SDK，下载链接如下：

- RK3576 Andriod14 SDK

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_U/manifests -b rk3576 -m
Android14.xml
```

- 其他芯片的Android14 SDK

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_U/manifests -b master -m
Android14.xml
```

4.5.2 编译说明

SSI编译分为system（Android16 AOSP）和 vendor（Android14 非AOSP部分），分别编译后再通过工具合并成一个完整的固件。使用rkbuild工具进行编译。

4.5.3 如何关闭GKI

SDK编译时会默认开启GKI配置，适用于需要过GMS/EDLA认证的产品。如果你的产品没有过GMS/EDLA认证的需求，且不需要使用GKI，可以参考如下配置关闭GKI：

Android16 SSI SDK上面修改

- 关掉GKI的宏

```
device/rockchip/rockchip_generic$
diff --git a/vendorsetup.sh b/vendorsetup.sh
index 42c7cbb..832f72f 100755
--- a/vendorsetup.sh
+++ b/vendorsetup.sh
@@ -177,7 +177,7 @@ export PRODUCT_BUILDING_GRF=true
 export PRODUCT_KERNEL_ARCH=arm64
 export BOARD_AVB_ENABLE_OVERRIDE=true
 export BOARD_USES_AB_IMAGE=true
-export BOARD_BUILD_GKI_OVERRIDE=true
+export BOARD_BUILD_GKI_OVERRIDE=false
 export PRODUCT_SHIPPING_API_LEVEL_OVERRIDE=$product_api_level
 export BOARD_SUPER_PARTITION_SIZE_OVERRIDE=$super_partition_size
 export BUILD_WITH_GOOGLE_MARKET_OVERRIDE=$build_with_google_market
```

- 关掉GMS包

```
device/rockchip/rockchip_generic$
diff --git a/ssi_16_arm64/ssi_16_arm64.mk b/ssi_16_arm64/ssi_16_arm64.mk
index ec99d08..0169db2 100644
--- a/ssi_16_arm64/ssi_16_arm64.mk
+++ b/ssi_16_arm64/ssi_16_arm64.mk
@@ -14,9 +14,9 @@ PRODUCT_BRAND := rockchip
 PRODUCT_MODEL := SSI 16 on ARM64
 PRODUCT_MANUFACTURER := rockchip

-BUILD_WITH_GOOGLE_MARKET ?= false
+BUILD_WITH_GOOGLE_MARKET := false
 BUILD_WITH_GOOGLE_MARKET_ALL ?= false
-BUILD_WITH_GOOGLE_GMS_EXPRESS ?= false
+BUILD_WITH_GOOGLE_GMS_EXPRESS ?= true
 BUILD_WITH_EEA ?= false
 BUILD_WITH_EEA_TYPE ?= v2_type4c
```

- 修改super分区大小

修改BOARD_SUPER_PARTITION_SIZE宏的值，具体位置如下：

```
device/rockchip/rockchip_generic/configs/dynamic_partitions.mk
BOARD_SUPER_PARTITION_SIZE ?= 5872025600
```

注意：BOARD_SUPER_PARTITION_SIZE修改的值需要与Android14 SDK里面配置的值一样，否则可能编译失败或者无法开机。

4.5.4 编译工具rkbuild

- 在Android16 SSI的SDK中执行source build/envsetup.sh命令后会创建rkbuild相关工具链接

```
Android16_SSI_SDK$ source build/envsetup.sh

Android16_SSI_SDK$ rkbuild
rkbuild     rkbuild_build     rkbuild_merge     rkbuild_mkupdate     rkbuild_pathmod
   rkbuild_reset     rkbuild_setup     rkbuild_status     rkbuild_usage
```

- rkbuild的源码位置：

```
device/rockchip/rockchip_generic/vendorsetup.sh
```

- rkbuild使用说明

```
@sys2206:~/d1_Android16_ssi$ rkbuild_usage
Usage: -bash { setup | reset | status | pathmod | build | help }

- setup { -d <kernel_dts> -l <vendor_lunch> -f <build_number> -p
<vendor_sdk_path> }

    Setup the Android_VENDOR_SDK software link

- reset

    Reset the Android_VENDOR_SDK software link

- status

    Query the Android_VENDOR_SDK software link status

- pathmod

    Print the source directory corresponding to the module name
    pathmod libfoo[.vendor], libfoo.vendor will look for libfoo in the GRF SDK

- merge

    Merge images from target-files

- mkupdate
```

```
    Create update.img

- build { uboot | kernel | ssi | vendor | all }

    - uboot: Build the uboot in Android_VENDOR_SDK

    - kernel: Build the kernel modules in Android_VENDOR_SDK

    - ssi: Build the system SDK

    - vendor: Build the GRF SDK

    - all: Build the uboot & kernel & ssi & vendor, then pack the images to
update.img

- help
```

4.5.5 编译步骤

- 确认已经下载了Android16 SSI SDK和Android14 SDK。
- 进入到Android16 SSI SDK根目录下配置编译环境

```
@sys2206:~/d1_Android16_ssi$ source build/envsetup.sh
```

- 编译SSI的system

SSI的对应的device产品目录为：

```
device/rockchip/rockchip_generic/
```

对应的产品lunch定义在

```
device/rockchip/rockchip_generic/AndroidProducts.mk

 21 COMMON_LUNCH_CHOICES := \
 22     ssi_16_arm64-bp2a-userdebug \
 23     ssi_16_arm64-bp2a-user \
 24     ssi_16_arm64_go-bp2a-userdebug \
 25     ssi_16_arm64_go-bp2a-user
```

支持 ssi_16_arm64和ssi_16_arm64_go两种配置。
选择对应的lunch

```
@sys2206:~/d1_Android16_ssi$ lunch ssi_16_arm64-bp2a-userdebug

============================================
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=16
TARGET_PRODUCT=ssi_16_arm64
TARGET_BUILD_VARIANT=userdebug
TARGET_ARCH=arm64
```

```
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-76-generic-x86_64-Ubuntu-20.04.6-LTS
HOST_CROSS_OS=windows
BUILD_ID=BP2A.250605.031.A3
OUT_DIR=out
SOONG_ONLY=false
============================================

```~

  Wondering whether to use user, userdebug or eng?

  user   The builds that ship to users. Reduced debugability.
  userdebug   High fidelity to user builds but with some debugging options
              enabled. Best suited for performance testing or day-to-day use
              with debugging enabled.
  eng   More debugging options enabled and faster build times, but
              runtime performance tradeoffs. Best suited for day-to-day
              local development when not doing performance testing.

```~
```

开始编译system

- 方式一，使用make编译

```
@sys2206:~/d1_Android16_ssi$ make dist -j32
```

- 方式二，使用rkbuild脚本编译

```
@sys2206:~/d1_Android16_ssi$ rkbuild_build ssi
```

- 编译完整的固件

1). 链接Android14 SDK及配置编译参数

```
@sys2206:~/d1_Android16_ssi$ rkbuild_setup
请输入Android_VENDOR_SDK的目录(例如: /home/hcq/4_Android14_0409):
/home/wlq/b2_Android14_rk3576/  #这里输入Android14 SDKde 路径

[WARN] Android_VENDOR_SDK 可选 lunch 项目如下:

You're building on Linux

Lunch menu .. Here are the common combinations:

     1. rk3562_u-user

     2. rk3562_u-userdebug

     3. rk3562_ugo-user

     4. rk3562_ugo-userdebug
```

```
     5. rk3576_u-user

     6. rk3576_u-userdebug

     7. rk3576s_u-user

     8. rk3576s_u-userdebug

请输入Android_VENDOR_SDK的lunch选项(例如: rk3588_u-user): rk3576_u-serdebug   #这里选择Android14对应的lunch

需要指定build number? (回车自动生成, 或直接输入需要的build number):    #这里输入build number 可以不填直接回车即可

device/rockchip/rockchip_generic/configs/gms.mk:10: warning: Please set client id with your own MADA ID! now is android-rockchip
device/rockchip/rockchip_generic/modules/gms.mk:27: warning: Final select gms-mandatory_64bit_only and mainline_modules with android-rockchip
.
.
build/make/core/board_config.mk:241: warning: ------------------------->>>>>
TARGET_DEVICE_DIR is device/rockchip/rockchip_generic/ssi_16_arm64
~/b2_Android14_rk3576 ~/d1_Android16_develop

============================================
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=14
PRODUCT_INCLUDE_TAGS=com.android.mainline
TARGET_PRODUCT=rk3576_u
TARGET_BUILD_VARIANT=userdebug
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-76-generic-x86_64-Ubuntu-20.04.6-LTS
HOST_CROSS_OS=windows
BUILD_ID=UQ1A.240205.004.B1
OUT_DIR=out
============================================
~/d1_Android16_ssi
当前 PRODUCT_KERNEL_DTS=rk3576-evb1-v10, #若需要修改DTS请输入, 回车将使用默认值: 📄 正在解析模块路径:

/home/wlq/d1_Android16_ssi/.repo/manifests/include/rk_modules_repository.xml
[WARN] 软链接缓存文件将保存在: /home/wlq/d1_Android16_ssi/.symlinks_cache
[WARN] ⚠ 目标已存在但不是软链接，请检查是否有本地修改未提交:
/home/wlq/d1_Android16_ssi/bootable/recovery/pcba_core
/home/wlq/d1_Android16_ssi/vendor/rockchip/hardware/interfaces/neuralnetworks
[INFO] 🔄 删除已存在的软链接:
/home/wlq/d1_Android16_ssi/vendor/rockchip/hardware/interfaces/rksoundsetting
{
  "Android_SYSTEM_SDK": "/home/wlq/d1_Android16_develop",
  "Android_VENDOR_SDK": "/home/wlq/b2_Android14_rk3576",
  "system_lunch": "ssi_16_arm64-bp2a-userdebug",
  "system_product": "ssi_16_arm64",
  "system_variant": "userdebug",
  "system_release": "bp2a",
```

```
  "vendor_lunch": "rk3576_u-userdebug",
  "vendor_product": "rk3576_u",
  "vendor_variant": "userdebug",
  "platform_version": "Android16",
  "product_api_level": "36",
  "build_id": "BP2A.250605.031.A3",
  "super_partition_size": "5872025600",
  "build_with_google_market": "true",
  "build_with_google_gms_express": "false",
  "platform_soc": "rk3576",
  "uboot_config": "rk3576_defconfig gki.config",
  "kernel_config": "gki_defconfig rockchip_gki.config bifrost_gpu_gki.config
pcie_wifi.config",
  "kernel_dts": "rk3576-evb1-v10",
  "clang_version": "clang-r487747c",
  "kernel_path": "/home/wlq/b2_Android14_rk3576/kernel-6.1",
  "build_number": "202507",
  "symlink_cache": "/home/wlq/d1_Android16_ssi/.symlinks_cache",
  "grf_output": "/home/wlq/d1_Android16_ssi/IMAGE/GRF/ssi_16_arm64",
  "dist_direc": "/home/wlq/d1_Android16_ssi/IMAGE/GRF/ssi_16_arm64/dist",
  "images_dir": "/home/wlq/d1_Android16_ssi/IMAGE/GRF/ssi_16_arm64/rk3576_u-
userdebug"
}
```


以上编译步骤可以通过快捷命令完成如：

```
rkbuild setup -l {lunch_name} -d {dts_name} -p /home/wlq/b2_Android14_rk3576 -f
202506
```

其中：

- -l 是指定Android14的lunch名称
- -d 是指定kernel的dts
- -p 是指定Android14 SDK的路径
- -f 是指定build number，可以不设置

2). 开始编译

完整编译

```
@sys2206:~/d1_Android16_ssi$ rkbuild_build all
```

部分编译

```
- build { uboot | kernel | ssi | vendor | all }

    - uboot: Build the uboot in Android_VENDOR_SDK

    - kernel: Build the kernel modules in Android_VENDOR_SDK

    - ssi: Build the system SDK

    - vendor: Build the GRF SDK

    - all: Build the uboot & kernel & ssi & vendor, then pack the images to
update.img
```

如果是部分编译，编译完需要执行merge操作

```
@sys2206:~/d1_Android16_ssi$ rkbuild_merge
```

3). 编译后的固件路径

```
@sys2206:~/d1_Android16_ssi/rockdev/Image-rk3576_u$
```

4.5.6 固件说明

固件	源文件	说明
baseparameter.img	Android14 SDK	显示参数信息
boot.img	Android14 SDK	GKI的版本由google发布，包含内核核心部分；非GKI的由kernel-6.1源码编译，包含ramdisk、kernel、dtb
boot-debug.img	Android14 SDK	与boot.img的差别是user固件可以烧写这个boot.img进行root权限操作
dtbo.img	Android14 SDK	Device Tree Overlays 参考下面的dtbo章节说明
init_boot.img	Android14 SDK	GKI的版本才有，包含ramdisk的内容
config.cfg	Android14 SDK	烧写工具的配置文件，可以直接导入烧写工具显示需要烧写的选项
MiniLoaderAll.bin	Android14 SDK	一级loader，包含ddr、spl
misc.img	Android14 SDK	包含recovery-wipe开机标识信息，烧写后会进行recovery
parameter.txt	Android14 SDK	包含分区信息
pcba_small_misc.img	Android14 SDK	包含pcba开机标识信息，烧写后会进入简易版pcba模式
pcba_whole_misc.img	Android14 SDK	包含pcba开机标识信息，烧写后会进入完整版pcba模式
recovery.img	Android14 SDK	非GKI版本才有，包含recovery-ramdis、kernel、dtb
super.img	Android16 SDK + Android14 SDK	包含odm、product、vendor、system、system_ext分区内容，其中system、system_ext分区有Android16 SDK编译而来
uboot.img	Android14 SDK	由u-boot源码编译而来
vbmeta.img	Android14 SDK	包含avb校验信息，用于AVB校验
update.img	Android16 SDK + Android14 SDK	包含以上需要烧写的img文件，可以用于工具直接烧写整个固件包

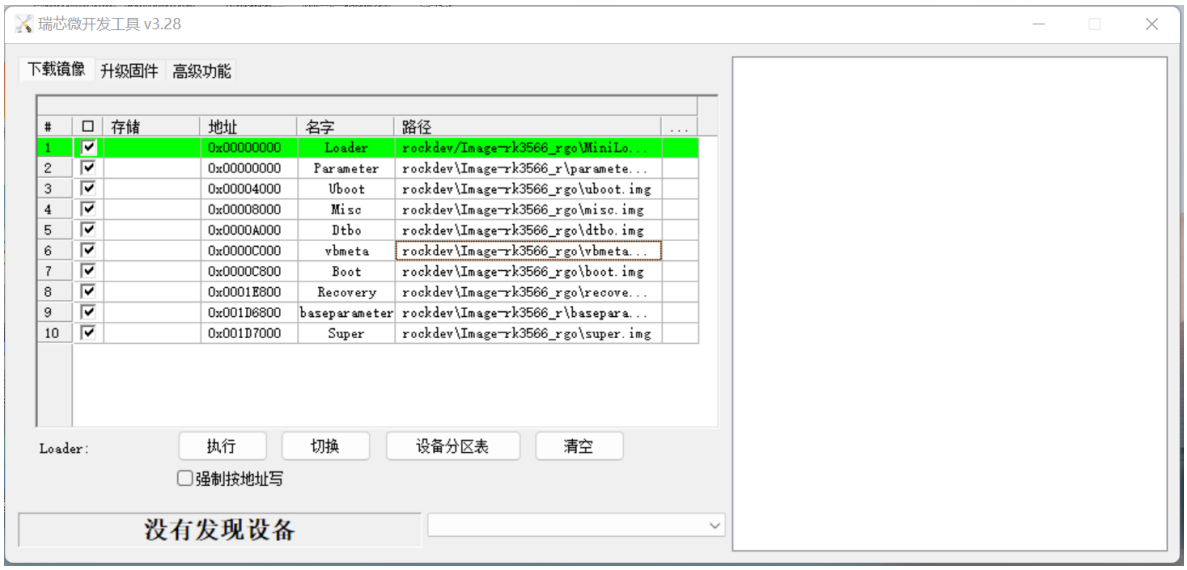
固件	源文件	说明
ssi_16_arm64-images-merged-fastboot.zip	Android16 SDK + Android14 SDK	由Android16和Andriod14编译后的素材包合并而来，可以用fastboot命令可烧写格式的固件包
ssi_16_arm64-ota-merged.zip	Android16 SDK + Android14 SDK	由Android16和Andriod14编译后的素材包合并而来，合并后的OTA升级完整包，用于OTA升级
ssi_16_arm64-target_files-merged.zip	Android16 SDK + Android14 SDK	由Android16和Andriod14编译后的素材包合并而来，合并后的素材包，可以用来做OTA的差分包

4.6 固件烧写

4.6.1 固件烧写工具

Android16的USB驱动DriverAssitant需要更新到V5.1.1版本，可以参考下面的工具章节进行更新。
Windows烧写工具：（工具是时刻更新，请及时同步更新）

```
RKTools/windows/AndroidTool/AndroidTool_Release_v3.28
```




```
RKTools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool_v2.30
```

在下文工具说明章节有详细说明

可用于烧写编译生成的散包固件和`update.img`

4.7 Generic Kernel Image（GKI）

Android14过GMS和EDLA认证的产品都强制kernel使用GKI，GKI的配置和编译具体参考文档RKDocs/android/Rockchip_Developer_Guide_Android14_GKI_CN.pdf

4.8 fastboot烧写动态分区

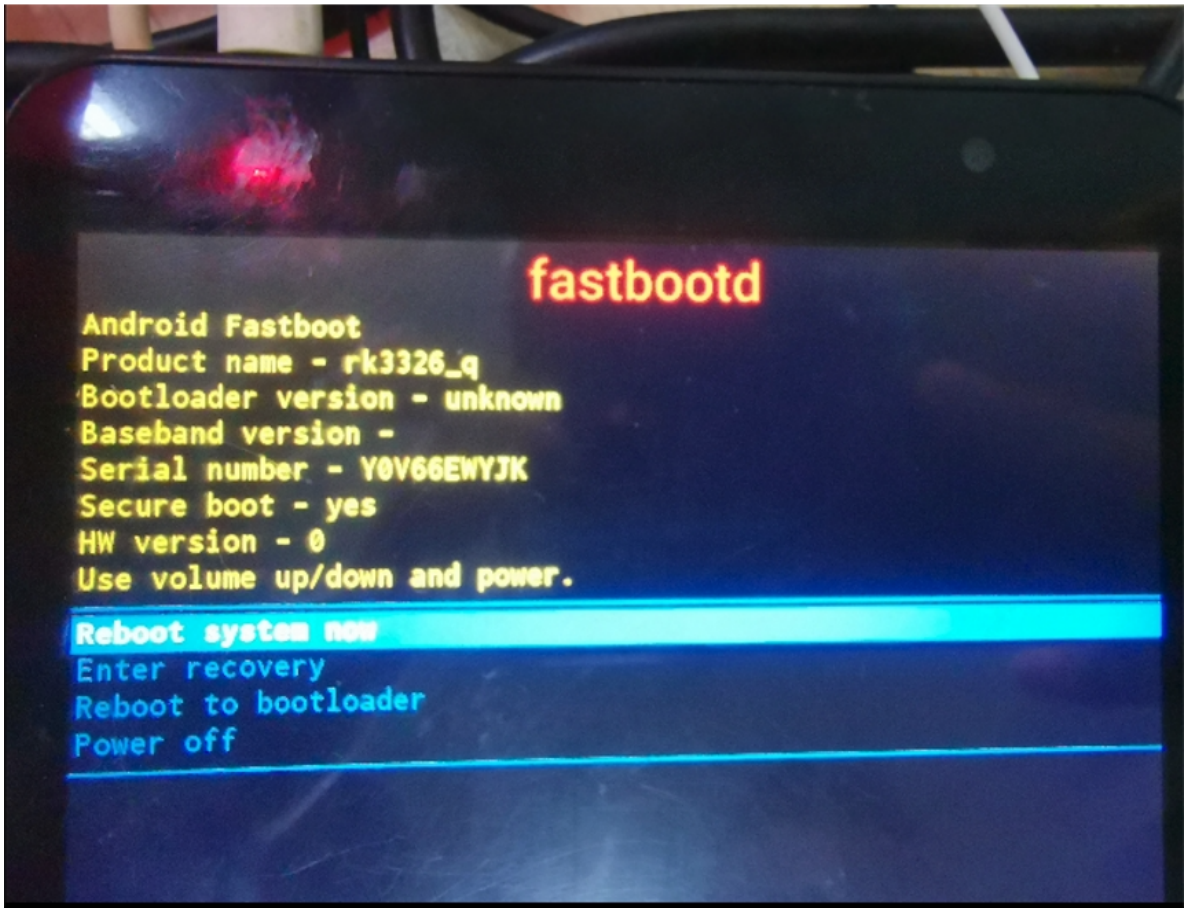
R的新设备支持动态分区，已经移除了system/vendor/odm/product/system_ext分区，请烧写super.img，单独烧写system/vendor/odm等（可以在out下面找到对应img文件）可以用`fastbootd`，要求adb和fastboot版本均为最新，SDK提供了编译好的工具包：

```
RKTools/linux/Linux_adb_fastboot (Linux_x86版本)
RKTools/windows/adb_fastboot (Windows_x86版本)
```

- 使用命令烧写动态分区：

```
adb reboot fastboot
fastboot flash vendor vendor.img
fastboot flash system system.img
fastboot flash odm odm.img
```

注：进入fastbootd模式后，屏幕上会显示相关设备信息，如图所示：



注：非动态分区使用**fastboot**，请进入**bootloader**:

```
adb reboot bootloader
```

烧写GSI的方法：

- 确认机器解锁后，进入fastbootd，只需要烧写GSI中的system.img及固件中的misc.img，烧写后会进入recovery进行恢复出厂设置。下面附上整个烧写流程：

1. 重启至bootloader，未解锁->机器解锁：

```
adb reboot bootloader
fastboot oem at-unlock-vboot ## 对于烧写过avb公钥的客户，请参考对应的文档解锁。
```

2. 恢复出厂设置，重启至fastbootd：

```
fastboot flash misc misc.img
fastboot reboot fastboot  ## 此时将进入fastbootd
```

3. 开始烧写GSI

```
fastboot delete-logical-partition product  ## (可选)对于分区空间紧张的设备，可以先执行本条命令删除product分区后再烧写GSI
fastboot flash system system.img
fastboot reboot  ## 烧写成功后，重启
```

- 注：也可以使用DSU(Dynamic System Updates)烧写GSI，目前Rockchip平台已经默认支持DSU。由于该功能需要消耗大量内存，不建议1G DDR及以下的设备使用，有关DSU的说明和使用，请参考Android官网：

https://source.android.com/devices/tech/ota/dynamic-system-updates

- 注1：VTS测试时，需要同时烧写编译出的boot-debug.img到boot分区；
- 注2：CTS-ON-GSI测试时则不需要烧boot-debug.img；
- 注3：测试时请使用Google官方发布的，带有-signed结尾的GSI镜像；

5. 使用DTBO功能

Android 10.0及以上支持Device Tree Overlays功能，开发过程体现在需要烧写dtbo.img，用于多个产品间的兼容等。
修改方法：

1. 找到(或指定)模板文件：

```
get_build_var PRODUCT_DTBO_TEMPLATE
```

例如：

```
PRODUCT_DTBO_TEMPLATE := $(LOCAL_PATH)/dt-
overlay.in(device/rockchip/rk388/rk3588_u/dt-overlay.in)
```

2. 添加或修改需要的节点：

例如：

```
/dts-v1/;
/plugin/;
```

```
&chosen {
    bootargs_ext = "androidboot.boot_devices=${_boot_device}";
};

&firmware_android {
    vbmeta {
        status = "disabled";
    };
    fstab {
        status = "disabled";
    };
};

&reboot_mode {
    mode-bootloader = <0x5242C309>;
    mode-charge = <0x5242C30B>;
    mode-fastboot = <0x5242C303>;
    mode-loader = <0x5242C301>;
    mode-normal = <0x5242C300>;
    mode-recovery = <0x5242C303>;
};
```

注: 使用dtbo时一定要确保dts中存在alias，否则无法成功overlay

6. 修改fstab文件

1. 找到(或指定)模板文件：

```
get_build_var PRODUCT_FSTAB_TEMPLATE
```

例如：

```
PRODUCT_FSTAB_TEMPLATE := device/rockchip/common/scripts/fstab_tools/fstab.in
```

2. 修改：添加分区挂载、修改swap_zram参数，修改data分区格式等

7. 修改parameter.txt

Android 14添加了生成parameter.txt的工具，支持根据配置参数编译出parameter.txt。如果没有配置模板文件，则会寻找添加修改好的parameter.txt文件。

1. 找到(或指定)模板文件：

```
get_build_var PRODUCT_PARAMETER_TEMPLATE
```

例如：

```
PRODUCT_PARAMETER_TEMPLATE :=
device/rockchip/common/scripts/parameter_tools/parameter.in
```

2. 修改配置分区大小(例如)：

```
BOARD_SUPER_PARTITION_SIZE := 2688548864
BOARD_DTBOIMG_PARTITION_SIZE := xxxx
BOARD_BOOTIMAGE_PARTITION_SIZE := xxxxx
BOARD_CACHEIMAGE_PARTITION_SIZE := xxxx
```

3. 不使用parameter生成工具：

添加一个parameter.txt文件到你的device目录下即可：
例如：device/rockchip/rk3326/rk3326_u/parameter.txt

4. 仅使用工具生成parameter.txt(例如)：

```
parameter_tools --input
device/rockchip/common/scripts/parameter_tools/parameter.in --firmware-version
16.0 --machine-model rk3326 --manufacturer rockchip --machine rk3326_u --
partition-list
uboot_a:4096K,trust_a:4M,misc:4M,dtbo_a:4M,vbmeta_a:4M,boot_a:33554432,backup:30
0M,security:4M,cache:300M,metadata:4096,frp:512K,super:2G --output
parameter_new.txt
```

8. Android14 SDK

其他部分文档请参考Android14 SDK中的开发指南
《RK3576_Developer_Guide_Android14_SDK_CN.pdf》
《Rockchip_Developer_Guide_Android14_SDK_CN.pdf》。

9. 附录 A 编译开发环境搭建 Compiling and development environment setup

9.1 Initializing a Build Environment

This section describes how to set up your local work environment to build the Android source files. You must use Linux or Mac OS; building under Windows is not currently supported.
For an overview of the entire code-review and code-update process, see Life of a Patch.
Note: All commands in this site are preceded by a dollar sign ($) to differentiate them from output or entries within files. You may use the Click to copy feature at the top right of each command box to copy all lines without the dollar signs or triple-click each line to copy it individually without the dollar sign.

9.2 Choosing a Branch

Some requirements for the build environment are determined by the version of the source code you plan to compile. For a full list of available branches, see Build Numbers. You can also choose to download and build the latest source code (called master), in which case you will simply omit the branch specification when you initialize the repository.
After you have selected a branch, follow the appropriate instructions below to set up your build environment.

9.3 Setting up a Linux build environment

These instructions apply to all branches, including master.
The Android build is routinely tested in house on recent versions of Ubuntu LTS (14.04) and Debian testing. Most other distributions should have the required build tools available.
For Gingerbread (2.3.x) and newer versions, including the master branch, a 64-bit environment is required. Older versions can be compiled on 32-bit systems.
Note: See Requirements for the complete list of hardware and software requirements, then follow the detailed instructions for Ubuntu and Mac OS below.

9.4 Installing the JDK

The master branch of Android in the Android Open Source Project (AOSP) comes with prebuilt versions of OpenJDK below prebuilts/jdk/ so no additional installation is required.
Older versions of Android require a separate installation of the JDK. On Ubuntu, use OpenJDK. See JDK Requirements for precise versions and the sections below for instructions.
For Ubuntu >= 15.04
Run the following:

```
sudo apt-get update
sudo apt-get install openjdk-8-jdk
```

For Ubuntu LTS 14.04
There are no available supported OpenJDK 8 packages for Ubuntu 14.04. The Ubuntu 15.04 OpenJDK 8 packages have been used successfully with Ubuntu 14.04. Newer package versions (e.g. those for 15.10, 16.04) were found not to work on 14.04 using the instructions below.

1. Download the .deb packages for 64-bit architecture from old-releases.ubuntu.com:

```
openjdk-8-jre-headless_8u45-b14-1_amd64.deb with SHA256
0f5aba8db39088283b51e00054813063173a4d8809f70033976f83e214ab56c0
openjdk-8-jre_8u45-b14-1_amd64.deb with SHA256
9ef76c4562d39432b69baf6c18f199707c5c56a5b4566847df908b7d74e15849
openjdk-8-jdk_8u45-b14-1_amd64.deb with SHA256
6e47215cf6205aa829e6a0a64985075bd29d1f428a4006a80c9db371c2fc3c4c
```

2. Optionally, confirm the checksums of the downloaded files against the SHA256 string listed with each package above. For example, with the sha256sum tool:

```
sha256sum {downloaded.deb file}
```

3. Install the packages:

```
sudo apt-get update
```

Run dpkg for each of the .deb files you downloaded. It may produce errors due to missing dependencies:

```
sudo dpkg -i {downloaded.deb file}
```

To fix missing dependencies:

```
sudo apt-get -f install
```

Update the default Java version - optional
Optionally, for the Ubuntu versions above update the default Java version by running:

```
sudo update-alternatives --config java
sudo update-alternatives --config javac
```

Note: If, during a build, you encounter version errors for Java, see Wrong Java version for likely causes and solutions.
Installing required packages (Ubuntu 14.04)
You will need a 64-bit version of Ubuntu. Ubuntu 14.04 is recommended.

```
sudo apt-get install git-core gnupg flex bison gperf build-essential zip curl
zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 lib32ncurses5-dev x11proto-
core-dev libx11-dev lib32z-dev ccache libgl1-mesa-dev libxml2-utils xsltproc
unzip python-pyelftools python3-pyelftools device-tree-compiler libfdt-dev
libfdt1 libssl-dev liblz4-tool python-dev
```

Note: To use SELinux tools for policy analysis, also install the python-networkx package. Note: If you are using LDAP and want to run ART host tests, also install the libnss-sss:i386 package.

9.5 Configuring USB Access

Under GNU/linux systems (and specifically under Ubuntu systems), regular users can't directly access USB devices by default. The system needs to be configured to allow such access.
The recommended approach is to create a file /etc/udev/rules.d/51-android.rules (as the root user) and to copy the following lines in it. must be replaced by the actual username of the user who is authorized to access the phones over USB.

```
# adb protocol on passion (Rockchip products)
SUBSYSTEM=="usb", ATTR{idVendor}=="2207", ATTR{idProduct}=="0010", MODE="0600",
OWNER="<username>"
```

Those new rules take effect the next time a device is plugged in. It might therefore be necessary to unplug the device and plug it back into the computer.
This is known to work on both Ubuntu Hardy Heron (8.04.x LTS) and Lucid Lynx (10.04.x LTS). Other versions of Ubuntu or other variants of GNU/linux might require different configurations.
References : http://source.android.com/source/initializing.html

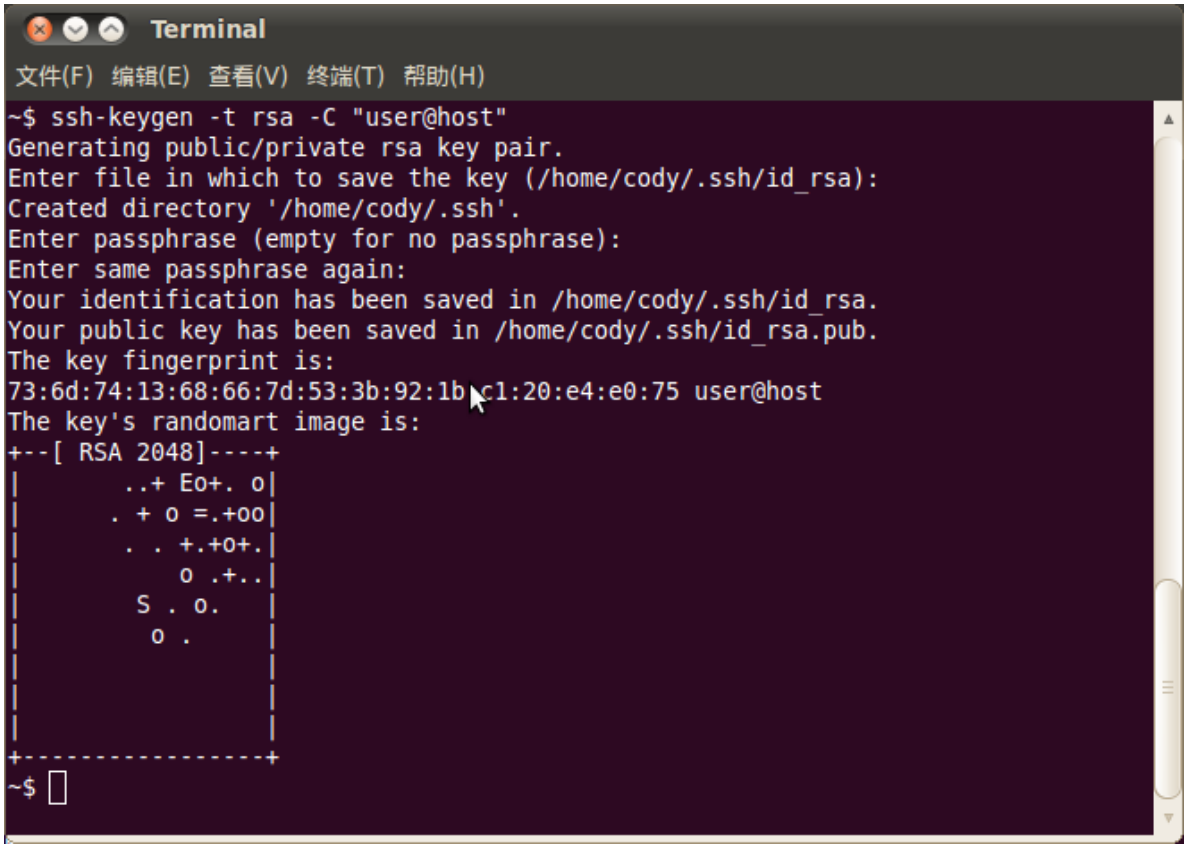
10. 附录 B SSH公钥操作说明 SSH public key operation instruction

10.1 附录 B-1 SSH公钥生成 SSH public key generation

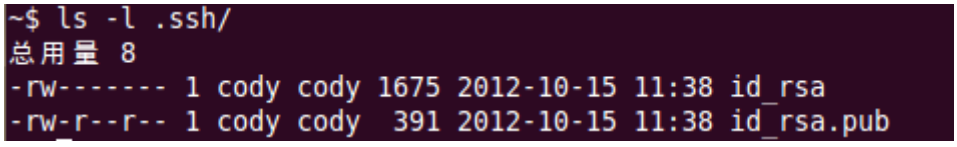
使用如下命令生成：

```
ssh-keygen -t rsa -C "user@host"
```

请将user@host替换成您的邮箱地址。



命令运行完成会在你的目录下生成key文件。



请妥善保存生成的私钥文件id_rsa和密码，并将id_rsa.pub发邮件给SDK发布服务器的管理员。

10.2 附录 B-2 使用key-chain管理密钥 Use key-chain to manage the key

推荐您使用比较简易的工具keychain管理密钥。
具体使用方法如下：

1. 安装keychain软件包：

```
$sudo aptitude install keychain
```

2. 配置使用密钥：

```
$vim ~/.bashrc
```

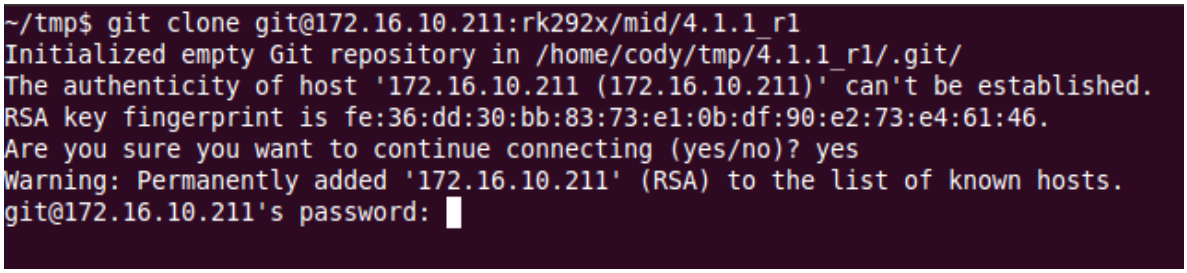
增加下面这行：

```
eval `keychain --eval ~/.ssh/id_rsa`
```

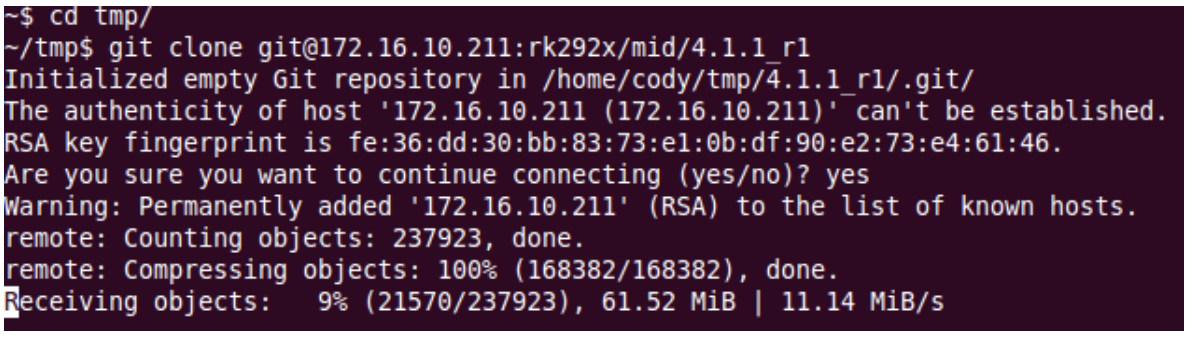
其中，id_rsa是私钥文件名称。
以上配置以后，重新登录控制台，会提示输入密码，只需输入生成密钥时使用的密码即可，若无密码可不输入。
另外，请尽量不要使用sudo或root用户，除非您知道如何处理，否则将导致权限以及密钥管理混乱。

10.3 附录 B-3 多台机器使用相同ssh公钥 Multiple devices use the same ssh public key

在不同机器使用，可以将你的ssh私钥文件id_rsa拷贝到要使用的机器的“~/.ssh/id_rsa”即可。
在使用错误的私钥会出现如下提示，请注意替换成正确的私钥。



添加正确的私钥后，就可以使用git 克隆代码，如下图。



添加ssh私钥可能出现如下提示错误。

```
Agent admitted failture to sign using the key
```

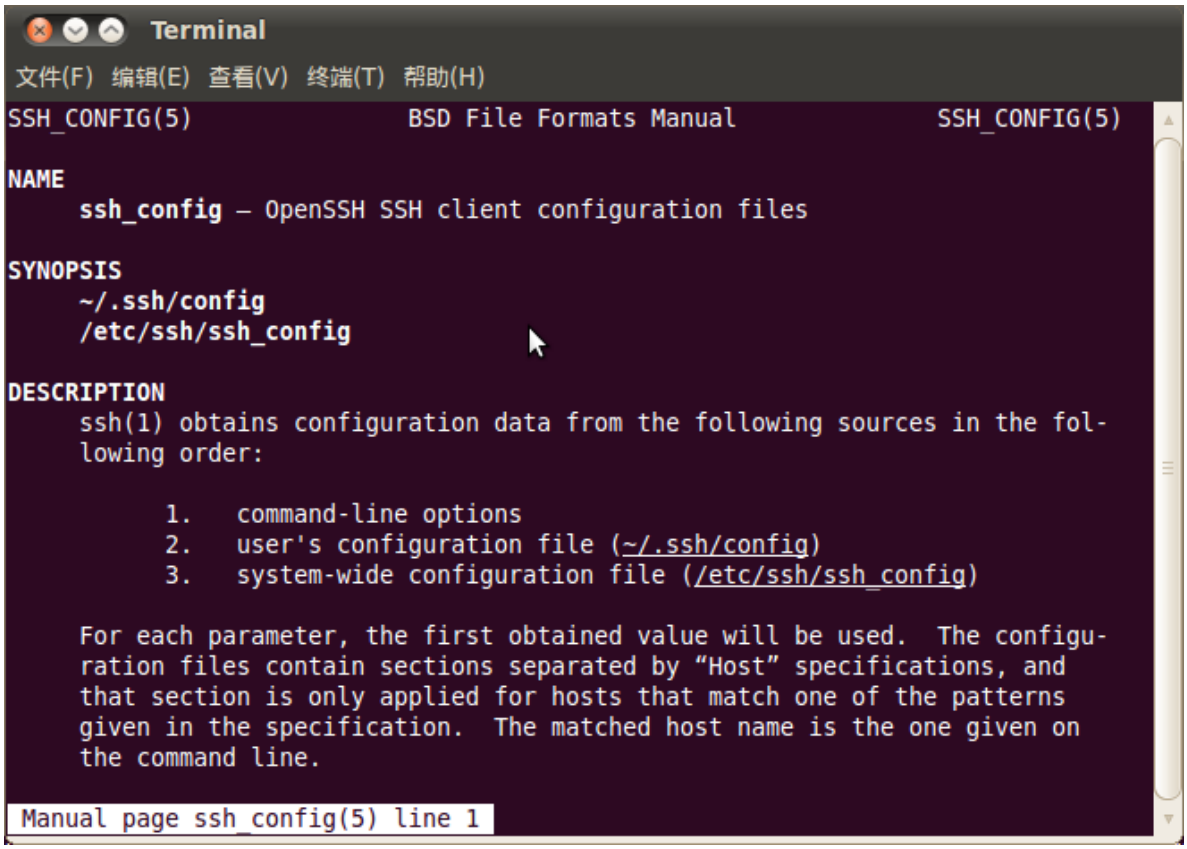
在console输入如下命令即可解决。

```
ssh-add ~/.ssh/id_rsa
```

10.4 附录 B-4 一台机器切换不同ssh公钥 Switch different ssh public keys on one device

可以参考ssh_config文档配置ssh。

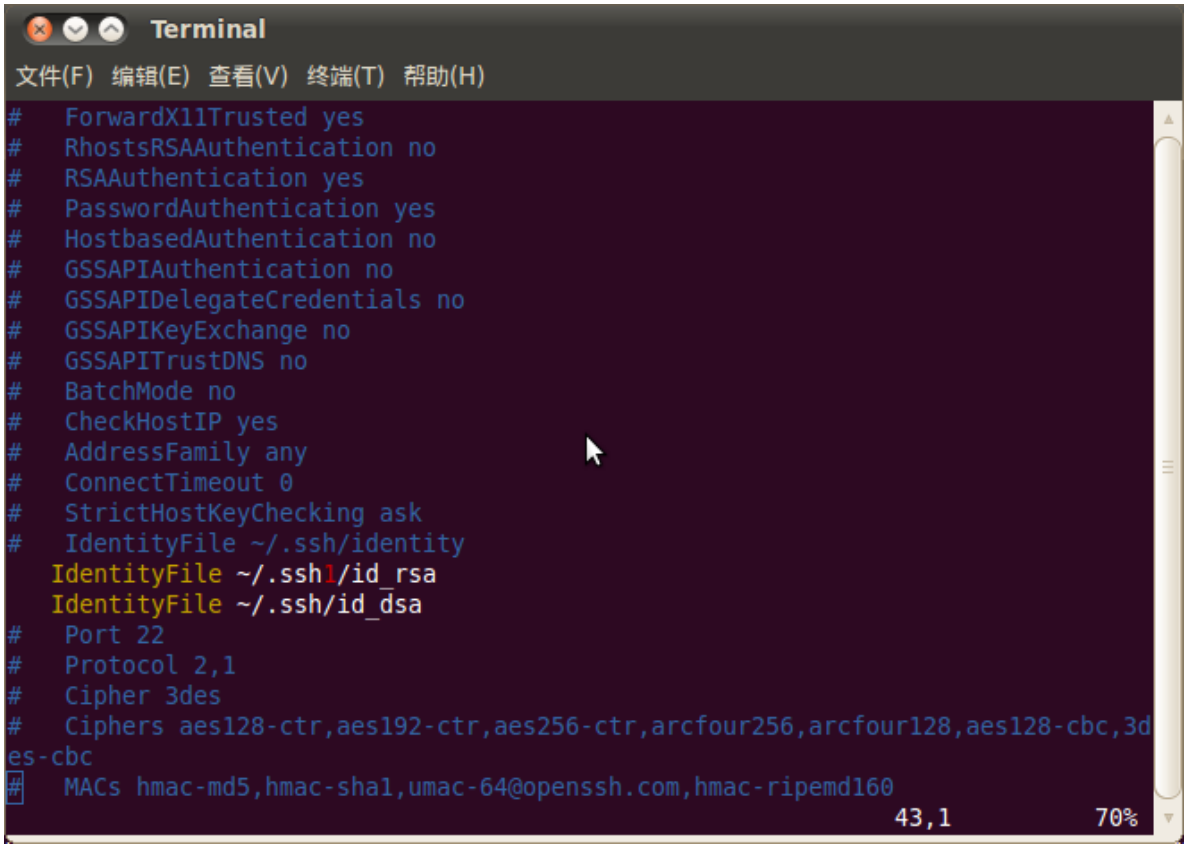
```
~$ man ssh_config
```



通过如下命令，配置当前用户的ssh配置。

```
~$ cp /etc/ssh/ssh_config ~/.ssh/config
~$ vi .ssh/config
```

如图，将ssh使用另一个目录的文件“~/.ssh1/id_rsa”作为认证私钥。通过这种方法，可以切换不同的密钥。



10.5 附录 B-5 密钥权限管理 Key authority management

服务器可以实时监控某个key的下载次数、IP等信息，如果发现异常将禁用相应的key的下载权限。
请妥善保管私钥文件。并不要二次授权与第三方使用。

10.6 附录 B-6 Git权限申请说明 Git authority application instruction

参考上述章节，生成公钥文件，发邮件至 fae@rock-chips.com，申请开通SDK代码下载权限。